

TheKnife



MANUALE TECNICO

Mattia Rotteri
Versione 1.0

Sommario

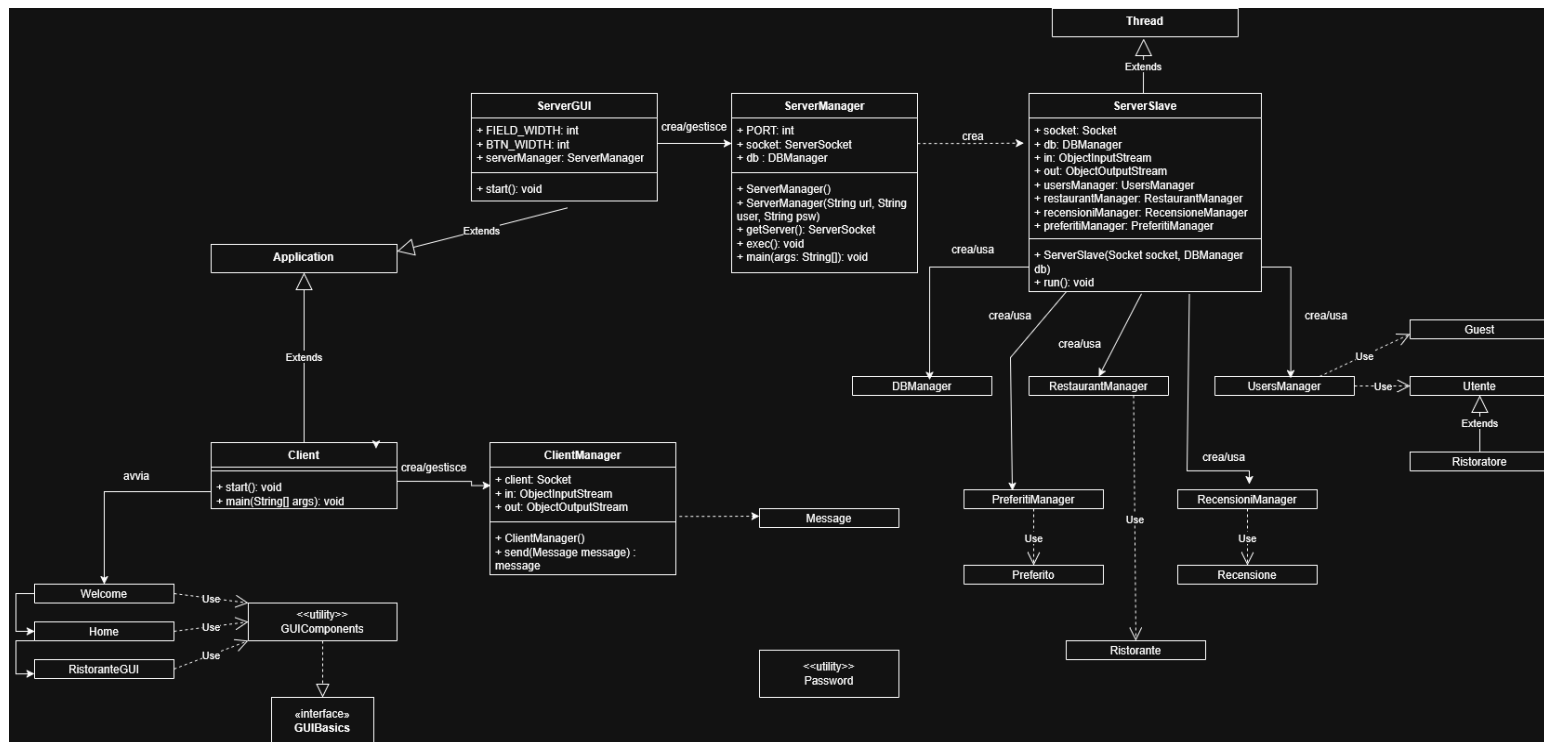
Sommario	1
Gerarchia delle classi.....	2
Diagrammi UML	3
Class Diagram	3
Sequence Diagram Registrazione/Login	4
Sequence Diagram Filtraggio	5
Sequence Diagram Recensione	5
Sequence Diagram Inserimento Ristorante	6
Schema ER	6
Gestione della connessione client-server.....	7
ServerSlave	7
Message	7
ClientManager	9
Database	10
DBManager	10
Gestione della connessione	10
Dataset di test	10
Modelli	10
Utenti dell'applicazione	11
Utente (Cliente)	11
Ristoratore	11
Guest (Ospite).....	12
Ristorante	12
Luogo	12
Recensione.....	12
Preferito.....	13
Managers	13
Sicurezza dei dati.....	14
Password.....	14
Limiti della soluzione sviluppata	14
Bibliografia e sitografia.....	14

Gerarchia delle classi

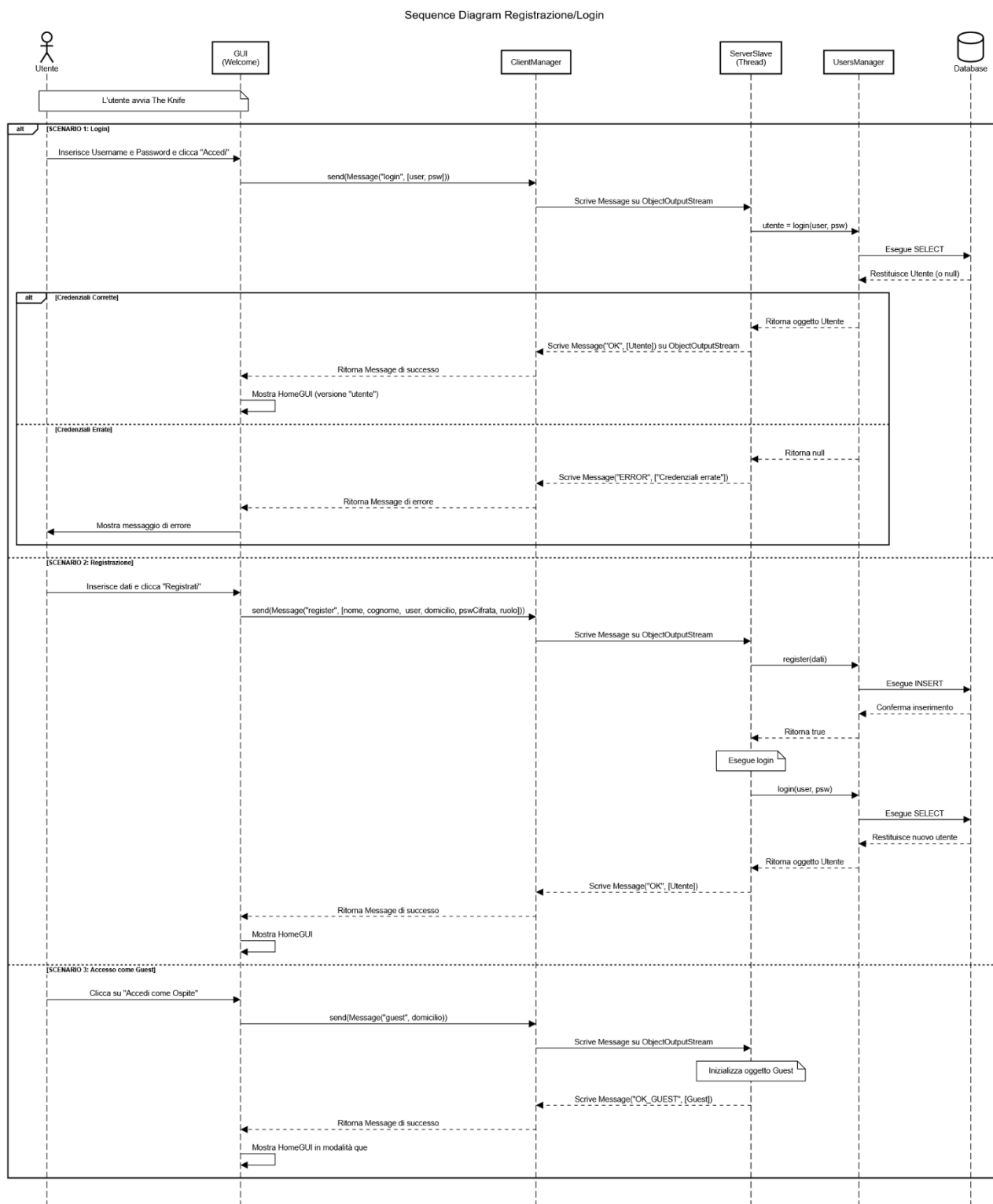
```
java.lang.Object
├── javafx.application.Application
│   ├── theknife.client.Client
│   └── theknife.server.gui.ServerGUI
├── theknife.client.ClientManager
├── theknife.server.DBManager
├── theknife.server.models.Guest (implements java.io.Serializable)
├── theknife.client.gui.GUIComponents (implements theknife.client.gui.GUIBasics)
├── theknife.client.gui.Home (implements theknife.client.gui.GUIBasics)
├── theknife.server.models.Luogo (implements java.io.Serializable)
├── theknife.Message (implements java.io.Serializable)
├── theknife.server.models.Password (implements java.io.Serializable)
├── theknife.server.PreferitiManager
├── theknife.server.models.Preferito (implements java.io.Serializable)
├── theknife.server.models.Recensione (implements java.io.Serializable)
├── theknife.server.RecensioneManager
├── theknife.server.RestaurantManager
├── theknife.server.models.Ristorante (implements java.io.Serializable)
├── theknife.client.gui.RistoranteGUI (implements theknife.client.gui.GUIBasics)
├── theknife.server.ServerManager
├── java.lang.Thread (implements java.lang.Runnable)
│   └── theknife.server.ServerSlave
├── theknife.server.UsersManager
├── theknife.server.models.Utente (implements java.io.Serializable)
│   └── theknife.server.models.Ristoratore
└── theknife.client.gui.Welcome (implements theknife.client.gui.GUIBasics)
```

Diagrammi UML

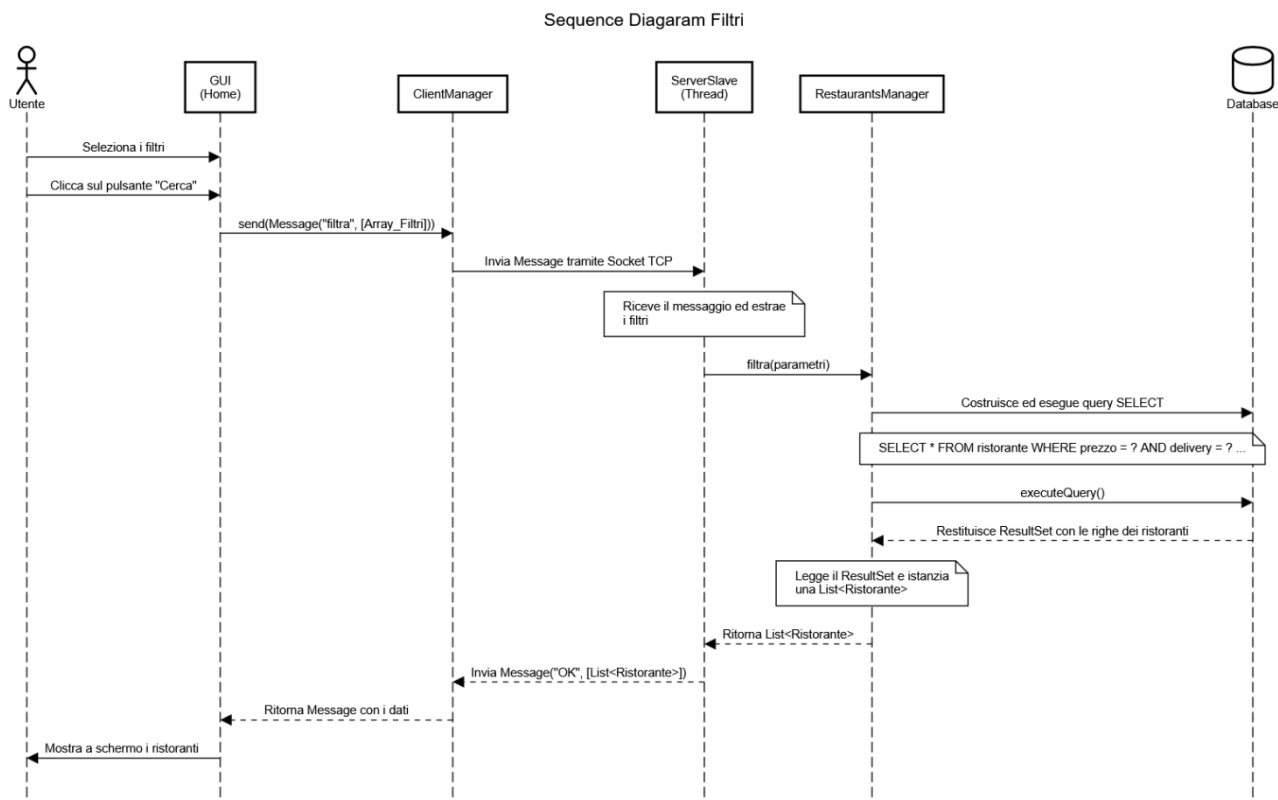
Class Diagram



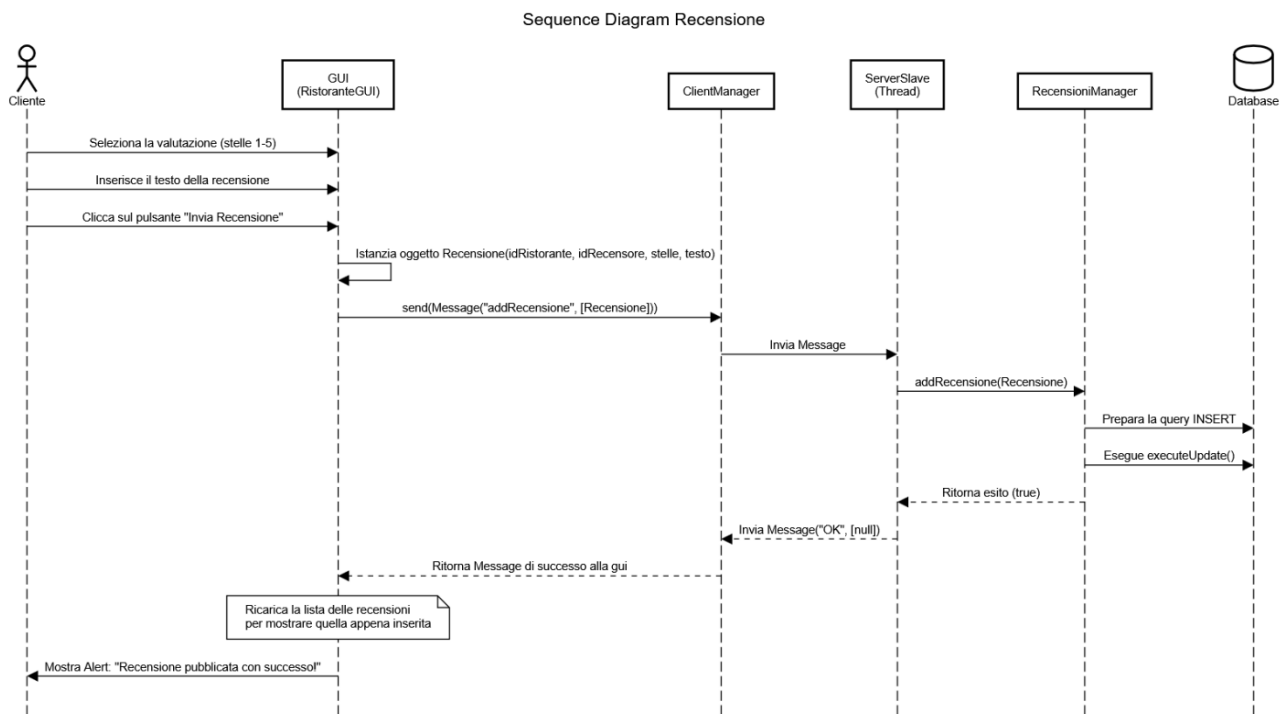
Sequence Diagram Registrazione/Login



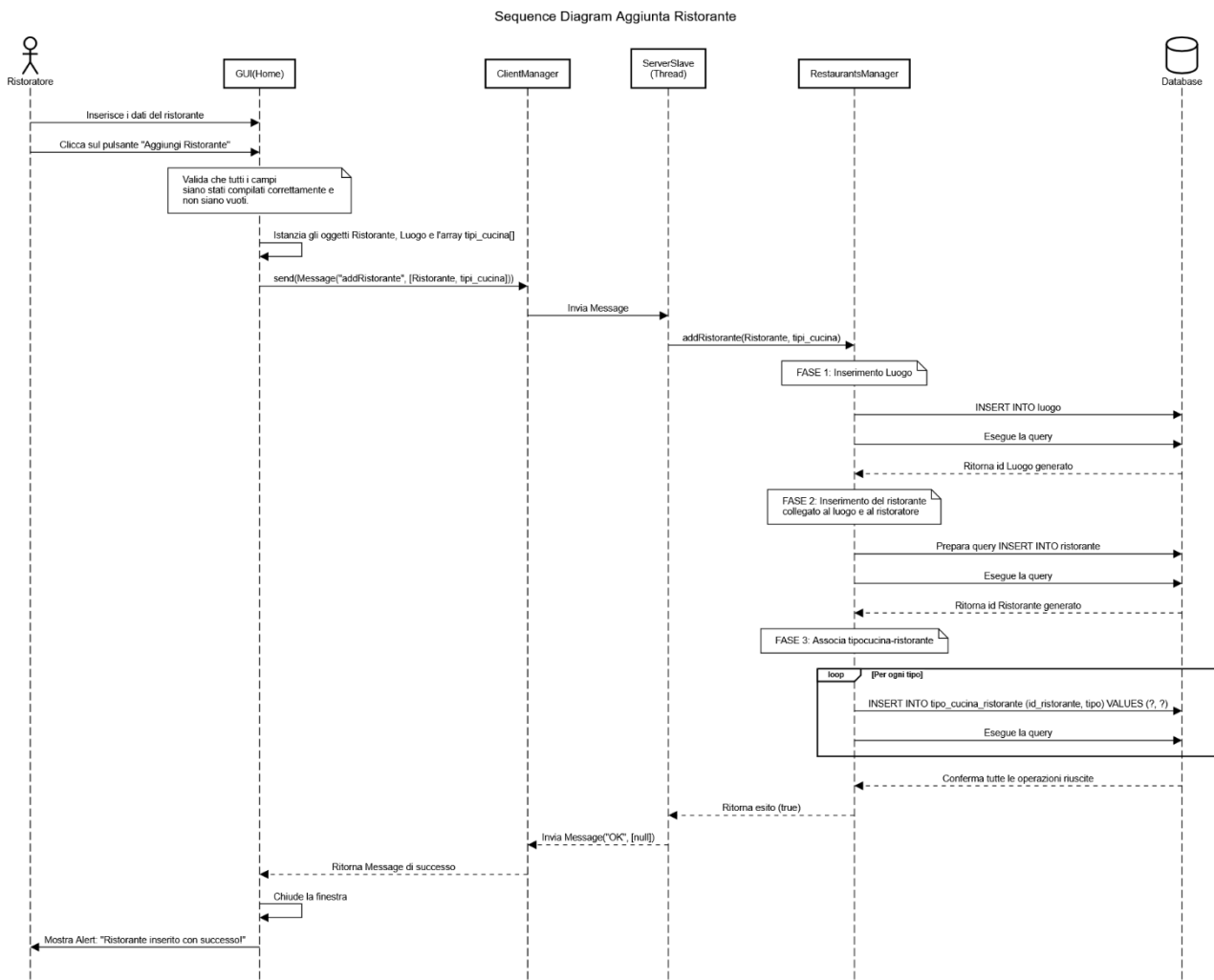
Sequence Diagram Filtraggio



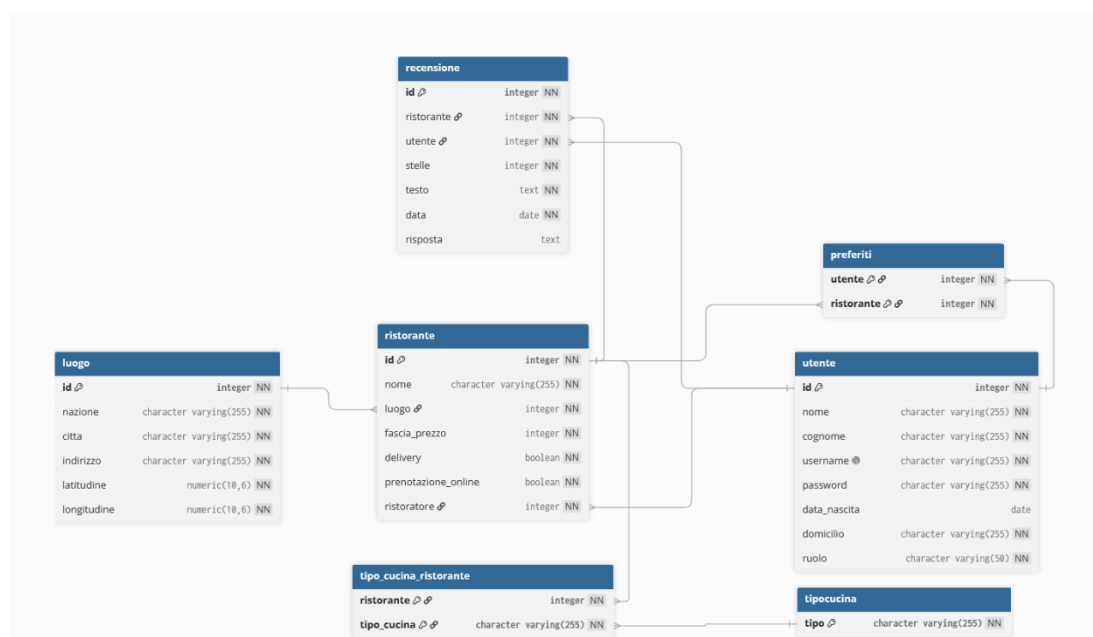
Sequence Diagram Recensione



Sequence Diagram Inserimento Ristorante



Schema ER



Gestione della connessione client-server

L'architettura di comunicazione di The Knife è basata su una connessione client-server tramite Socket. L'architettura è concorrente, ciò significa che il server non si blocca una volta ricevuta una connessione da parte di un client. Il `ServerManager` si occupa di rimanere in attesa di connessioni, una volta ricevuta una richiesta di connessione crea un nuovo Thread, `ServerSlave`, passandogli come parametri nel costruttore sia il Socket appena ottenuto, sia il riferimento al manager del database (`DBManager`).

ServerSlave

Il client parla esclusivamente con il `ServerSlave` a lui assegnato. Ogni `ServerSlave` presenta:

- **UsersManager:** si occupa della gestione delle operazioni relative agli utenti (login, registrazione).
- **PreferitiManager:** si occupa delle operazioni di gestione dei preferiti dell'utente (aggiunta preferito, rimozione preferito).
- **RestaurantManager:** si occupa delle operazioni relative ai ristoranti (aggiunta ristorante, eliminazione ristorante)
- **RecensioneManager:** si occupa delle operazioni di gestione delle recensioni (aggiunta, modifica, eliminazione recensione o risposta a una recensione).

Message

La classe `Message` è il pacchetto standardizzato usato da Client e Server per scambiarsi informazioni attraverso la rete. La classe implementa pertanto l'interfaccia `Serializable`.

Contiene due attributi fondamentali:

- **String op** (operazione): Indica l'operazione richiesta dal client oppure l'esito di un'operazione (OK oppure ERROR).
- **Object[] dati:** array di oggetti contenenti i parametri passati al server oppure il risultato dell'operazione richiesta.

Tabella delle operazioni:

Operazione (getOp())	Dati in Ingresso (getDati())	Descrizione operazione	Risposta del Server
<i>login</i>	[0]: Username (String), [1]: Password (String)	Controlla le credenziali utente tramite <code>UsersManager</code> .	OK con oggetto Utente o ERROR
<i>register</i>	[0] Nome, [1] Cognome, [2] Username, [3] Password, [4] Data	Registra un nuovo utente sul DB. Se l'operazione ha successo, esegue il login.	OK con oggetto Utente o ERROR

	Nascita, [5] Domicilio, [6] Ruolo (String)		
<i>guest</i>	[0]: domicilio (String)	Crea un oggetto Guest, per accesso ospite.	OK_GUEST con oggetto Guest
<i>getUtenteFromId</i>	[0]: Id Utente (int)	Recupera le informazioni di un utente cercandolo nel database tramite id e crea un oggetto Utente.	OK con oggetto Utente
<i>addRistorante</i>	[0]: Oggetto Ristorante, [1]: elenco tipi ristorante (String[])	Inserisce un nuovo luogo, ristorante e infine relazione ristorante-tipo nel database.	OK (nessun dato restituito)
<i>deleteRistorante</i>	[0]: Id del ristorante (int)	Rimuove un ristorante e tutte le relazioni associate a tale ristorante nel database.	OK o ERROR
<i>filtra</i>	[0..6]: Array di filtri e infine indirizzo dell'utente che vuole filtrare i ristoranti.	Applica i filtri passati tramite GUI alla lista di ristoranti, crea una lista di oggetti Ristorante filtrata.	OK con List<Ristorante>
<i>cercaFromNome</i>	[0]: Nome ristorante (String)	Cerca un ristorante nel database che corrisponda al nome passato come parametro.	OK con oggetto Ristorante o ERROR
<i>cercaFromId</i>	[0]: Id Ristorante (int)	Cerca un ristorante nel database che abbia come identificativo l'id passato come parametro.	OK con oggetto Ristorante o ERROR
<i>getTipoRistorante</i>	[0]: Id Ristorante (int)	Estrae tutti i tipi di cucina di un dato ristorante. Restituisce un array di stringhe contenente i tipi.	OK con List<String>
<i>getAllTipi</i>	Nessuno	Restituisce la lista completa di tipi di ristoranti.	OK con List<String>
<i>getRistorantiUtente</i>	[0]: Id Ristoratore (int)	Restituisce la lista di ristoranti di un dato utente proprietario.	OK con List<Ristorante>
<i>getPreferitiUtente</i>	[0]: Id Utente (int)	Restituisce la lista di ristoranti preferiti di un dato utente.	OK con List<Ristorante>
<i>addPreferiti</i>	[0]: Id Utente (int), [1]: Id Ristorante (int), [2]: Nome Ristorante (String)	Inserisce un nuovo record all'interno della tabella dei preferiti.	OK o ERROR

<i>removePreferiti</i>	[0]: Id Utente (int), [1]: ID Ristorante (int)	Cancella la relazione "preferiti" specificata dal database.	OK o ERROR
<i>recensioniRistorante</i>	[0]: Id Ristorante (int)	Restituisce la lista di Recensioni di un dato ristorante.	OK con List<Recensione>
<i>getRecensioniUtente</i>	[0]: Id Utente (int)	Restituisce la lista di recensione, lista di ristoranti, che un dato utente ha recensito.	OK con un array contenente List<Recensione> e List<Ristorante>
<i>infoRecensioni</i>	[0]: Id Ristorante (int)	Calcola le statistiche di un dato ristorante, numero medio di stelle e infine numero totale di recensioni.	OK con array di double[]
<i>rispondiRecensione</i>	[0]: Id Recensione (int), [1]: Testo (String)	Permette al ristorante di aggiungere una risposta testuale a una recensione ricevuta modificando l'apposito record.	OK o ERROR
<i>modifyRisposta</i>	[0]: Id Recensione (int), [1]: Nuovo testo (String)	Sovrascrive o aggiorna il testo della risposta precedentemente inserita dal ristorante.	OK o ERROR
<i>eliminaRisposta</i>	[0]: Id Recensione (int)	Pone a NULL l'attributo relativo alla risposta del ristorante nell'apposito record di Recensione.	OK o ERROR
<i>addRecensione</i>	[0]: Oggetto Recensione	Salva una recensione nel database.	OK o ERROR
<i>modifyRecensione</i>	[0]: Id Recensione (int), [1]: Nuovo testo (String)	Consente all'utente di modificare il testo di una recensione già pubblicata.	OK o ERROR
<i>removeRecensione</i>	[0]: Id Recensione (int)	Rimuove in modo definitivo la recensione dal database.	OK o ERROR

ClientManager

La classe ClientManager gestisce il socket lato client.

Il metodo di invio (send): La classe mette a disposizione un metodo che viene invocato dalle varie GUI ogni volta che l'utente clicca un bottone. Questo metodo:

- Prende l'oggetto Message preparato dalla GUI.
- Lo scrive sul flusso d'uscita (out.writeObject(request)).
- Restituisce un oggetto Message corrispondente alla risposta inviata dal server (letta tramite Message response = (Message) in.readObject()).

Database

Per garantire la persistenza e sicurezza di tutti i dati di *The Knife*, l'architettura si appoggia a un DBMS (Database Management System) Relazionale, nello specifico PostgreSQL.

DBManager

La classe DBManager si occupa di gestire la connessione con il database, tramite JDBC.

Gestione della connessione

- **Costruttore di Default:** Configurato per i test rapidi in locale (localhost). Tenta la connessione alla porta 5432 sul database theknife usando credenziali predefinite. *N.B. Non accessibile se non tramite codice, nella ServerGUI viene utilizzato il costruttore parametrizzato!*
- **Costruttore Parametrizzato:** Permette al server di connettersi ricevendo l'URL del database, l'username e la password.

Dataset di test

<https://www.kaggle.com/datasets/ngshiheng/michelin-guide-restaurants-2021>

Il dataset contiene informazioni dettagliate sui ristoranti di tutto il mondo inclusi nella Guida Michelin (aggiornato con i dati del 2021).

Il dataset è stato manipolato, estraendo solo informazioni utili al fine di *The Knife* e aggiungendo randomicamente un parametro intero per fascia di prezzo e valori booleani che indicano la possibilità di prenotare online oppure utilizzare il servizio di consegna a domicilio.

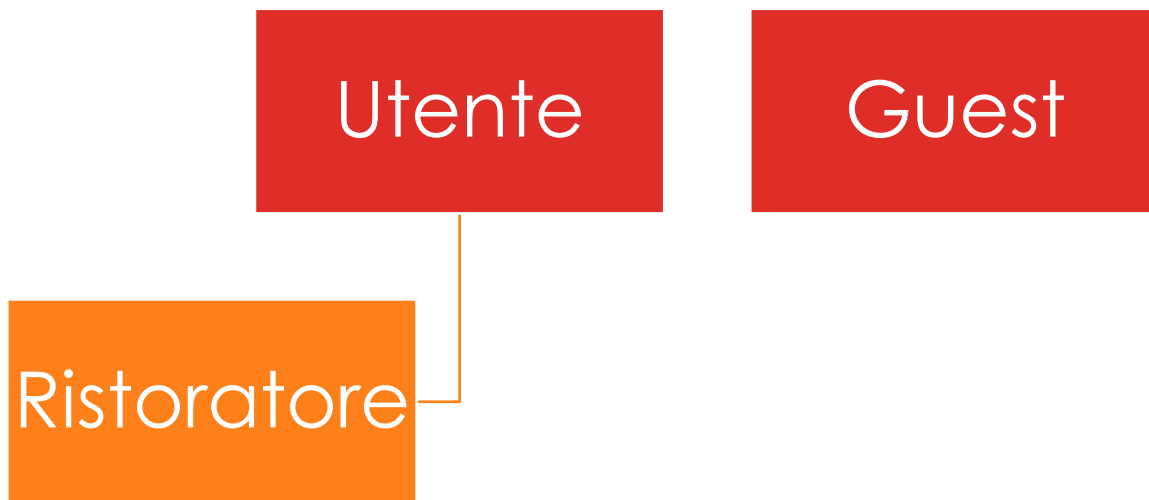
Modelli

Le classi contenute nel package *models* rappresentano gli oggetti della realtà usati per modellare il sistema SW e i record del Database.

Ogni classe presenta degli attributi corrispondenti ai campi delle tabelle del database, fornendo inoltre metodi getter e setters la loro lettura e scrittura.

Inoltre, implementano l'interfaccia *Serializable* per consentire il loro utilizzo come dati nello scambio di [Message](#).

Utenti dell'applicazione



Utente (Cliente)

Rappresenta l'utente che ha effettuato la registrazione come cliente. Una volta autenticato può svolgere le seguenti operazioni:

- Consultare la lista dei ristoranti.
- Filtrare la lista dei ristoranti
- Visualizzare i dettagli di un ristorante.
- Aggiungere ristoranti ai preferiti.
- Rimuovere ristoranti dai preferiti.
- Aggiungere una recensione a un ristorante.
- Eliminare o modificare una recensione.

Ristoratore

La classe Ristoratore estende la classe Utente, può pertanto svolgere tutte le operazioni svolte dal cliente, tuttavia può svolgere ulteriori operazioni:

- Creare un nuovo ristorante.
- Eliminare un proprio ristorante.
- Visualizzare le statistiche del proprio ristorante (media stelle e numero di recensioni).
- Rispondere alle recensioni.
- Eliminare/modificare recensioni.

Guest (Ospite)

La classe Guest rappresenta l'ospite anonimo, ovvero che ha deciso di non registrarsi/fare login e di conseguenza può utilizzare solo alcune funzionalità dell'applicazione. *N.B è necessario fornire il proprio domicilio!*

- Consultare la lista dei ristoranti.
- Filtrare la lista dei ristoranti.
- Visualizzare i dettagli di un ristorante.
- Leggere recensioni.

Ristorante

Rappresenta l'entità di Ristorante. Al suo interno contiene attributi riguardanti i dati del ristorante, come:

- Id → Id univoco del record del ristorante nel database.
- Nome (String)
- Luogo (Luogo)
- Prezzo medio (int)
- Delivery (boolean)
- Booking (boolean)

Luogo

Modella le informazioni geografiche e logistiche di un ristorante. Contiene i campi testuali per la geolocalizzazione (nazione, città, indirizzo) e le coordinate numeriche (latitudine e longitudine), per mappare la posizione dei ristoranti.

- Id (int) → Id univoco del record del Luogo nel database.
- Indirizzo (String)
- Città (String)
- Nazione (String)
- Latitudine (double)
- Longitudine (double)

Recensione

Rappresenta l'entità di una Recensione. Contiene gli attributi:

- IdRecensione (int) → Id univoco del record della recensione nel database.
- IdRecensore (int)
- IdRistorante (int)
- Testo (String) → Testo della recensione.
- Risposta (String) → Risposta del ristorante.

Preferito

Rappresenta la relazione preferito utente-ristorante. Contiene gli attributi:

- IdRistorante (int)
- IdUtente (int)
- NomeRistorante (String)

Managers

I *models* sono i contenitori dei dati, associato a ogni modello c'è un Manager (*UsersManager*, *PreferitiManager*, *RecensioneManager*, *RestaurantManager*) che si occupa di implementare la logica del server. Il loro compito è quello di fare da tramite tra Java e il database, ovvero occuparsi di tutte quelle operazioni riguardanti inserimenti, interrogazione, modifiche, eliminazioni nel DB.

I Manager sono gli unici componenti del sistema a contenere le istruzioni SQL (SELECT, INSERT, UPDATE, DELETE) ed operano un preciso ordine:

- **Ricezione della Connessione:** Tutte le classi ricevono come parametro del costruttore il riferimento al DBManager, sfruttando così un'unica connessione.
- **Uso di PreparedStatement:** La creazione della query nei metodi avviene tramite PreparedStatement e mai attraverso la concatenazione di stringhe, in modo da evitare problemi di sicurezza.
- **Risultato:** Nelle query *SELECT*, i risultati vengono letti riga per riga tramite *ResultSet*.

Esempio di utilizzo

```
public Ristorante cercaRistorante(int id) {
    String query = "SELECT ris.*, luogo.nazione, luogo.citta, luogo.indirizzo, luogo.latitudine,
    luogo.longitudine FROM ristorante ris JOIN luogo ON ris.luogo = luogo.id WHERE ris.id = ?";
    try(PreparedStatement statement = db.connection.prepareStatement(query)) {
        statement.setInt(1, id);
        try(ResultSet rs = statement.executeQuery()) {
            if(rs.next()) {
                Luogo luogo = new Luogo( rs.getString("indirizzo"), rs.getString("citta"),
                rs.getString("nazione"), rs.getDouble("latitudine"), rs.getDouble("longitudine"));
                return new Ristorante(rs.getInt("id"), rs.getString("nome"), luogo, rs.getInt("fascia_prezzo"),
                rs.getBoolean("delivery"), rs.getBoolean("prenotazione_online"), rs.getInt("ristoratore"));
            }
        }
    } catch(SQLException e) {}
    return null;
}
```

Sicurezza dei dati

La sicurezza delle credenziali d'accesso è gestita attraverso un'apposita classe Password per evitare la memorizzazione di password in chiaro all'interno del database.

Password

La classe implementa un algoritmo di cifratura molto simile al *Caesar Cipher*:

- **encrypt**(String psw): Ha come parametro la password in chiaro. Calcola la chiave (key = lunghezza della stringa) e trasla ogni singolo carattere in avanti di un numero di posizioni pari alla key.
- **Decrypt**(String encryptedValue): Esegue l'operazione inversa, data una password criptata, calcola la chiave e trasla ogni singolo carattere indietro di un numero di posizioni pari alla key.

Esempio di utilizzo

```
String clearValue = "Mattia"; //key = 6  
String pswEncrypted = Password.encrypt(passwordInChiaro); //psw cifrata: 'Sgzzog'
```

Limiti della soluzione sviluppata

- **Scalabilità:** Se l'app dovesse gestire contemporaneamente centinaia o migliaia di utenti, il server esaurirebbe le risorse.
- **Validazione dati semplificata:** vengono eseguiti controlli relativamente semplici sui dati.
- **Sicurezza:** Algoritmo di cifratura molto semplice e potenzialmente "vulnerabile".
- **Lingua:** L'applicazione è sviluppata interamente in italiano, tuttavia i dati presenti nel dataset sono in inglese.

Bibliografia e sitografia

- *Corso di Programmazione concorrente e distribuita*. Professore Luigi Lavazza.
- [API Nominatim](#)
- [The Absolute Beginners Guide to Reactive JavaFX](#)
- [API JavaFX](#)
- [CSS JavaFX](#)
- [JavaFX GeeksForGeeks](#)
- Video youtube relativi al design JavaFX